

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	Assessment:	Debugging and Optimisation task
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/nonrestoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	Kolar Umme Sadiya	Reg. No.:	192425117
Section / Batch:	3 rd year /AI&ML	Date:	04/08/2026

Instructions to Students

- Identify arithmetic errors and performance bottlenecks in the given problem.
- Analyse the execution steps to locate the source of errors.
- Debug the algorithm by correcting logical and computational faults.
- Optimize the solution using appropriate arithmetic techniques or algorithms.
- Evaluate the optimized solution for correctness, accuracy, and efficiency.

QUESTIONS

1. A digital signal processing system produces incorrect results during signed 8-bit integer addition using 2's complement representation when large positive numbers are added. Debug the binary addition process to determine whether overflow has occurred. Recommend an effective overflow detection and handling mechanism to ensure reliable computation.
2. A high-speed embedded controller experiences increased execution time because its arithmetic unit is built with a ripple carry adder. Debug the carry propagation process to identify the cause of the delay. Propose the use of a carry look-ahead adder and explain how it improves the overall arithmetic performance.
3. A processor's signed multiplication module generates incorrect outputs when multiplying negative binary numbers using Booth's algorithm. Debug the algorithm by examining bit-pair evaluation, arithmetic right shifts, and sign extension at each iteration. Identify the fault and recommend modifications to produce accurate multiplication results.
4. A microcontroller performing binary division exhibits poor performance because its restoring division implementation repeatedly restores the partial remainder after unsuccessful subtraction. Debug the sequence of operations to identify redundant steps. Suggest replacing it with the nonrestoring division algorithm and explain how the optimization reduces execution time.
5. A numerical computing application produces inaccurate floating-point results when combining values with widely different magnitudes using IEEE 754 single-precision representation. Debug the computation by examining exponent alignment, normalization, and rounding operations. Recommend suitable optimization techniques, such as higher precision or improved numerical methods, to enhance computational accuracy.

Code of Conduct

I Kolar Umme sadiya192425117 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Kolar Umme sadiya

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Identification of Errors / Bugs	20	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Marks Evaluation:

Q. No	Identification of Errors / Bugs (20)	Debugging Approach & Analysis (20)	Correctness of Debugged Solution (25)	Optimization Techniques Applied (20)	Testing and Documentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

1. Debugging 8-bit Signed Addition Using 2's Complement

A digital signal processing system produces incorrect results during signed 8-bit integer addition using 2's complement representation when large positive numbers are added.

Example

Consider the addition of +100 and +50.

+100 = 01100100

+50 = 00110010

Result = 10010110

The binary result 10010110 represents -106 in 2's complement, which is incorrect because the actual sum is 150. The valid range of an 8-bit signed integer is -128 to $+127$, so overflow has occurred.

Overflow Detection

The processor detects overflow when:

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.
- The carry into the Most Significant Bit (MSB) is different from the carry out of the MSB.

Overflow Handling

To ensure reliable computation:

- Set the Overflow Flag (V).
- Generate an exception or warning.
- Use a larger data type such as 16-bit or 32-bit integers.
- Apply saturation arithmetic in DSP applications to limit the result within the valid range.

Conclusion

An effective overflow detection mechanism prevents incorrect arithmetic results and improves the reliability of the digital signal processing system.

2. Debugging Carry Propagation in a Ripple Carry Adder

A high-speed embedded controller experiences increased execution time because its arithmetic unit uses a Ripple Carry Adder (RCA).

Cause of Delay

In a Ripple Carry Adder, each full adder waits for the carry generated by the previous stage before producing its output.

Bit0 $\rightarrow$ Bit1 $\rightarrow$ Bit2 $\rightarrow$ Bit3 $\rightarrow$ Bit4 $\rightarrow$ Bit5 $\rightarrow$ Bit6 $\rightarrow$ Bit7

This sequential carry propagation increases the overall delay, especially for larger adders.

Solution: Carry Look-Ahead Adder (CLA)

A Carry Look-Ahead Adder computes carry bits simultaneously using Generate (G) and Propagate (P) signals.

Generate Signal:

$$G_i = A_i \times B_i$$

Propagate Signal:

$$P_i = A_i \oplus B_i$$

Carry Equations:

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1G_0 + P_1P_0C_0$$

$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

Advantages of CLA

- Parallel carry generation.
- Faster addition.
- Reduced execution time.
- Improved processor performance.
- Suitable for high-speed embedded systems.

Conclusion

Replacing the Ripple Carry Adder with a Carry Look-Ahead Adder significantly improves arithmetic speed by eliminating sequential carry propagation.

3. Debugging Booth's Algorithm for Signed Multiplication

A processor generates incorrect outputs while multiplying negative binary numbers using Booth's algorithm.

Possible Faults

- Incorrect evaluation of the bit pair (Q_0, Q_{-1}).
- Improper arithmetic right shift.
- Missing sign extension.
- Incorrect initialization of the Booth bit (Q_{-1}).

Booth's Decision Table

$Q_0 Q_{-1}$ Operation

- | | |
|----|--------------|
| 00 | No operation |
| 01 | $A = A + M$ |
| 10 | $A = A - M$ |
| 11 | No operation |

Debugging Procedure

1. Verify the bit pair (Q_0, Q_{-1}) during every iteration.
2. Perform arithmetic right shifts while preserving the sign bit.
3. Ensure proper sign extension for negative operands.
4. Initialize the Booth bit (Q_{-1}) to 0 before starting the algorithm.
5. Repeat the process until all bits are processed.

Recommended Modifications

- Use arithmetic right shifts instead of logical shifts.
- Preserve the sign bit during every shift.
- Perform correct sign extension.
- Evaluate Booth bit pairs accurately.

Conclusion

Correct bit-pair evaluation, arithmetic right shifts, and sign extension ensure accurate signed multiplication using Booth's algorithm.

4. Debugging Restoring Division

A microcontroller performing binary division experiences poor performance because the restoring division algorithm repeatedly restores the partial remainder after unsuccessful subtraction.

Problem

During restoring division:

1. Shift the registers.
2. Subtract the divisor.
3. If the remainder becomes negative:
 - Restore the previous remainder.
 - Set quotient bit to 0.
4. Otherwise:
 - Keep the remainder.
 - Set quotient bit to 1.

Repeated restoration increases execution time.

Solution: Non-Restoring Division

The Non-Restoring Division algorithm avoids immediate restoration.

Algorithm:

- If the previous remainder is positive, subtract the divisor.
- If the previous remainder is negative, add the divisor in the next iteration.
- Continue without restoring the previous remainder.

Advantages

- Eliminates unnecessary restore operations.
- Reduces arithmetic operations.
- Faster execution.
- Better processor utilization.
- Improved computational efficiency.

Conclusion

Replacing restoring division with non-restoring division removes redundant restoration steps, reduces execution time, and improves overall performance.

5. Debugging IEEE 754 Floating-Point Computation

A numerical computing application produces inaccurate floating-point results when combining values with widely different magnitudes using IEEE 754 single-precision representation.

Causes of Inaccuracy

- Exponent alignment shifts the smaller mantissa to the right.
- Significant digits of the smaller number are lost.
- Normalization introduces rounding errors.
- Single precision provides only about 7 decimal digits of accuracy.

Example

$1000000.0 + 0.0001$

During exponent alignment, the smaller value is shifted significantly and may be lost because of limited mantissa precision.

Debugging Steps

1. Verify exponent alignment.
2. Check mantissa shifting.
3. Normalize the result correctly.
4. Apply IEEE 754 Round-to-Nearest-Even rounding.
5. Verify the final floating-point representation.

Optimization Techniques

- Use double precision (64-bit) instead of single precision.
- Apply numerically stable algorithms.
- Minimize repeated rounding operations.

- Reduce loss of significance during subtraction.
- Use higher precision for scientific and engineering computations.

Conclusion

Using higher precision and improved numerical methods significantly reduces floating-point errors, improves numerical stability, and enhances computational accuracy.